
Oliver Krauss

Transactions and Indices

RDB | MTD.ba VZ SS26

Hagenberg 08.04.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Single User Database Process

Tasks are handled sequentially. No problems with data consistency occur:

`SELECT * FROM AchievementTypes`



`INSERT INTO AchievementTypes VALUES (Collectibles, 22)`



`UPDATE AchievementTypes SET Points=40 WHERE AchievementType='Story'`



Time

Multi User Database Process

Parallel approach for queries is more efficient, and allows multiple users:

SELECT * FROM AchievementTypes



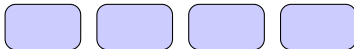
INSERT INTO AchievementTypes VALUES (Collectibles, 22)



UPDATE AchievementTypes SET Points=40 WHERE AchievementType="Story"



SELECT * FROM AchievementTypes



SELECT * FROM AchievementTypes



Time

Multi User Database Process

Parallel approach for queries is more efficient, and allows multiple users:

SELECT * FROM AchievementTypes



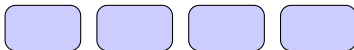
INSERT INTO AchievementTypes VALUES (Collectibles, 22)



UPDATE AchievementTypes SET Points=40 WHERE AchievementType="Story"



SELECT * FROM AchievementTypes



What is this SELECT reading?

SELECT * FROM AchievementTypes



Time

Transactions

- Group a sequence of operations as one logical unit
- Ensure **Data Integrity** via the ACID principle
- Enable **Concurrency** for multiple users or processes with conflict resolution
- Ensure **Failure recovery** against data corruption due to system crashes or errors.

ACID

Transactions in relational databases follow the ACID principle:

Atomicity Transactions are "all or nothing". Either every operation is completed, or none are.

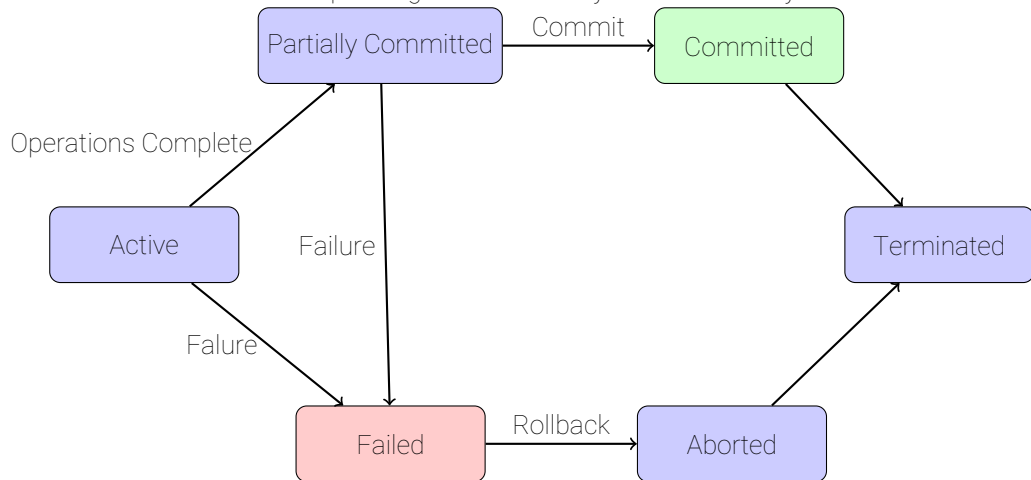
Consistency The data is always consistent. I.e. all changes must uphold all rules and constraints.

Isolation Transactions are executed independent of each other and do not interfere.

Durability A COMMITTED transaction is permanent. The changes remain in the database even if a failure occurs.

Transaction states

Transactions have a state depending on how far they are in their lifecycle:



Transaction Types

Read Only Transactions only read data but don't modify.

Read Write Transactions that are allowed to modify the data in addition to reading.

Anomalies in Transactions

Dirty read occurs when transaction receives data that has been changed by another transaction that has not committed yet.

Non-repeatable read happens when a transaction reads the same data multiple times, but that row is updated by another transaction that commits in between.

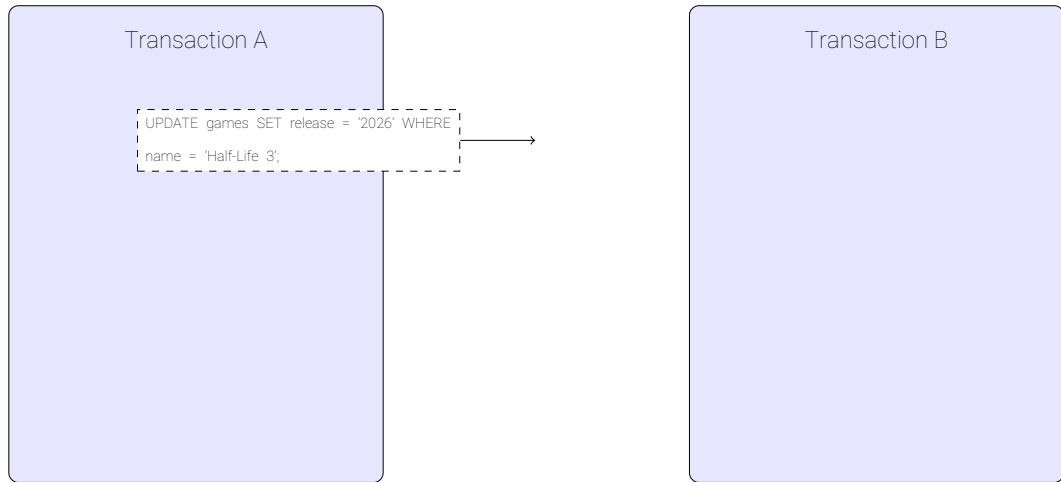
Phantom read occurs when a transaction reads several rows multiple times and rows are removed or added by other transaction that commit in between.

Dirty Read

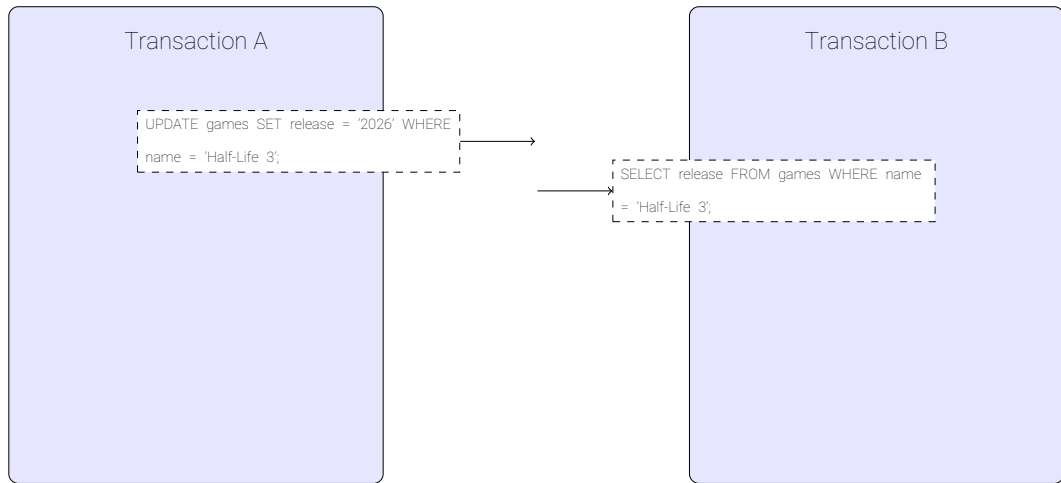
Transaction A

Transaction B

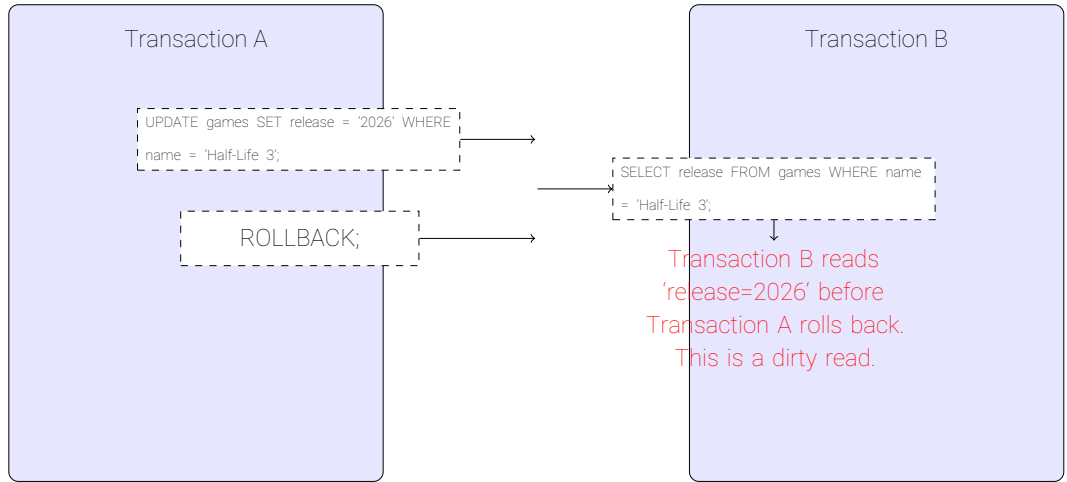
Dirty Read



Dirty Read



Dirty Read

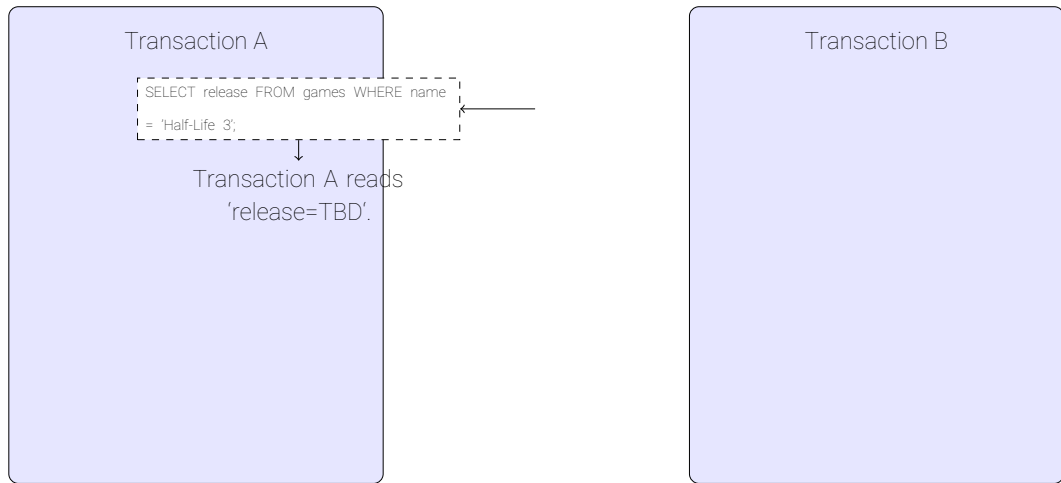


Non-Repeatable Read

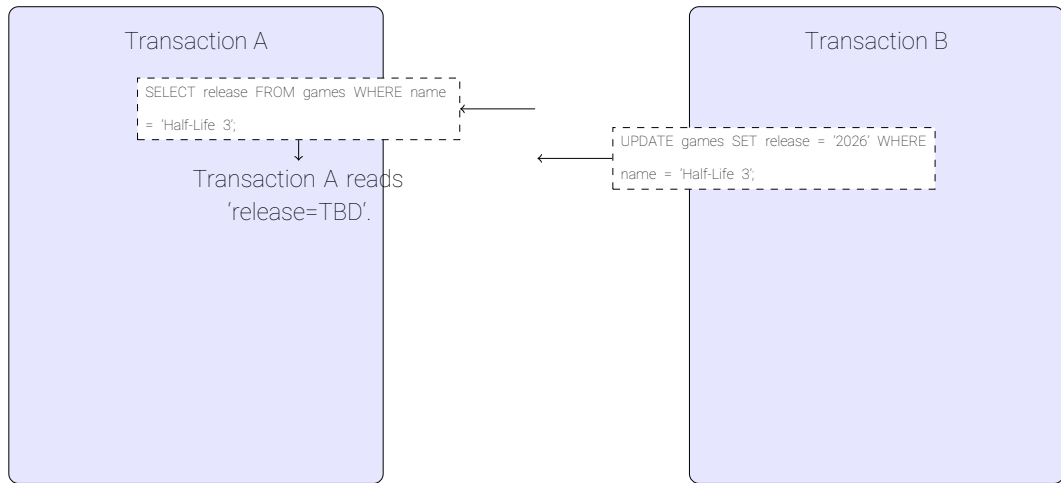
Transaction A

Transaction B

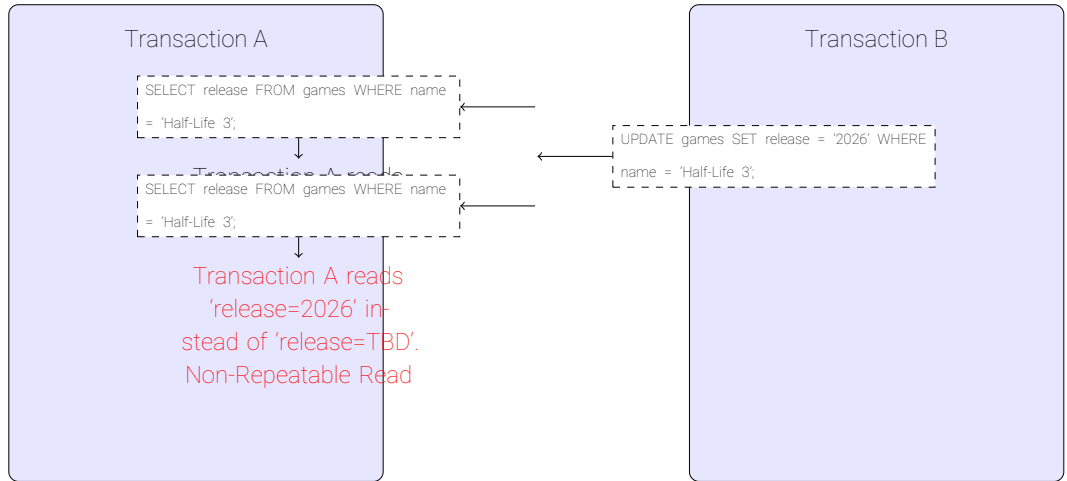
Non-Repeatable Read



Non-Repeatable Read



Non-Repeatable Read

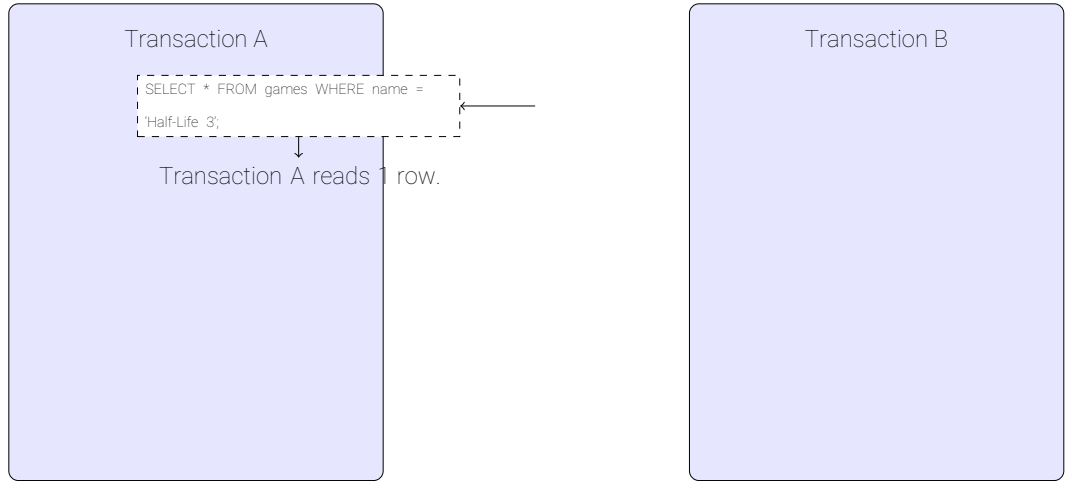


Phantom Read

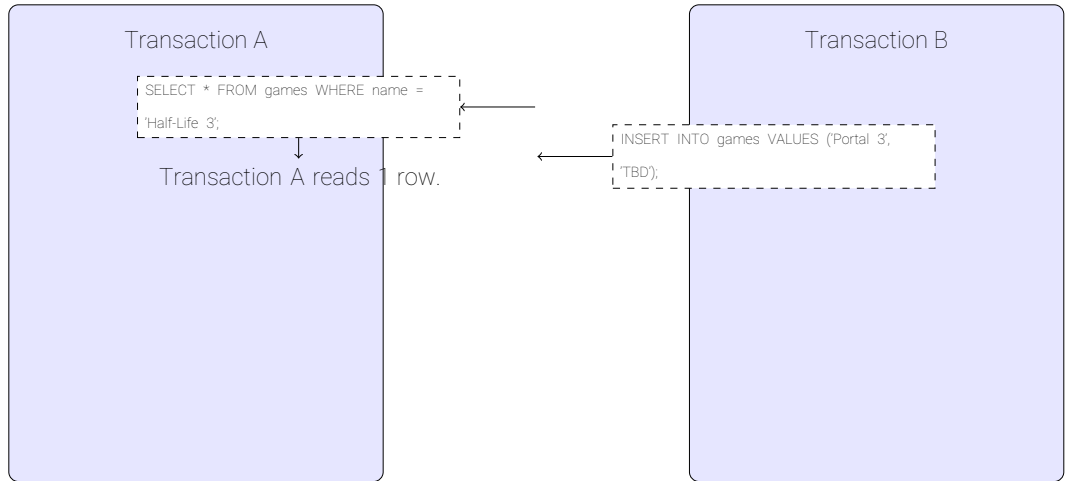
Transaction A

Transaction B

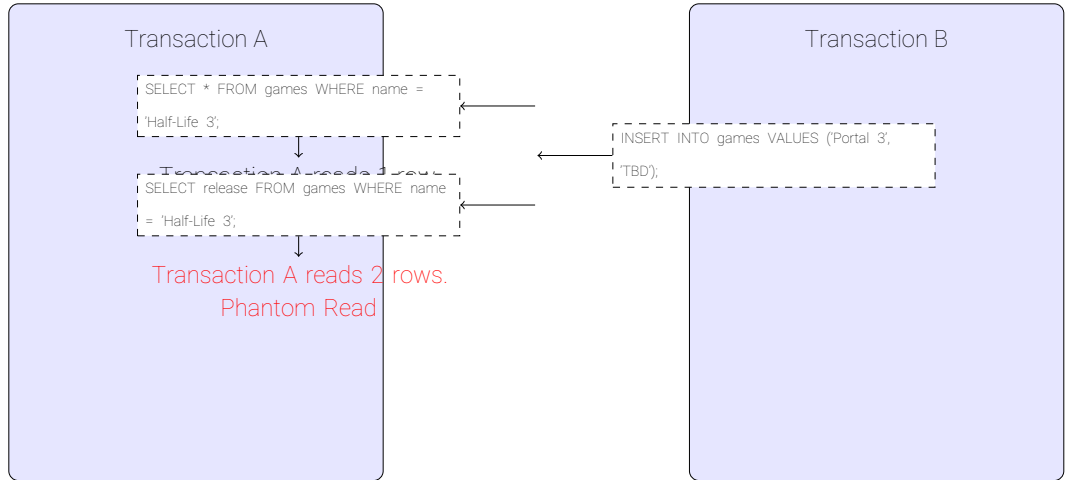
Phantom Read



Phantom Read



Phantom Read



Isolation Levels

To prevent anomalies databases offer isolation levels for transactions. In order from least to highest isolation:

Read uncommitted a transaction can read all data, even if not yet committed. All anomalies may occur.

Read committed allows a transaction to read only data that was committed. Dirty reads cannot occur.

Repeatable Read enforces read and write locks only on rows involved in the transaction, thus phantom reads may still occur.

Serializable enforces locks including ranges of the data queried (e.g. `Select *` locks a table, `Select * WHERE release = "TBD"` only locks rows with TBD)

Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Allowed	Allowed	Allowed
Read Committed	Prevented	Allowed	Allowed
Repeatable Read	Prevented	Prevented	Allowed
Serializable	Prevented	Prevented	Prevented

Table: Isolation Level to Anomaly Matrix

Concurrency Control Mechanisms

Databases have several options for concurrency control. Among them:

Locking The data that a transaction writes or reads to is locked until the transaction is finished.

Serialization Graph Checking Actions by transactions are ordered in a graph and dependency cycles are aborted.

Timestamp Ordering Checks if a transaction read/write conflicts with another transaction by the timestamp the data has, and aborts on errors.

Concurrency Control

Databases (often) apply a mixture of mechanisms and apply them optimistically or pessimistically:

Optimistic Concurrency Assumes minimal conflicts and checks consistency only at the commit of the transaction.

Pessimistic Concurrency Locks data needed for the transaction at the beginning

Mixed Methods Most real world databases use a mixture of both approaches

Deadlocks

Deadlocks occur when two or more transactions are waiting for each other to release a database entry (or table). This creates a circular dependency where no transaction can finish.:

- Deadlock Detection
 - Transactions are checked by analyzing the transaction graphs
 - Detected Deadlocks are solved by aborting one of the transactions (the victim)
 - Usually transactions are prioritized based on different factors (criticality, how long they are running, ...)
- Deadlock Prevention
 - Timeouts may be used to abort a transaction if it is taking too long (presumed deadlock)
 - Lock Ordering - transaction locks are ordered beforehand to avoid Deadlocks
 - Two Phase Locking (and other locking methods) - in the first phase locks are acquired and none released. In the second phase locks can only be released but none acquired.

Deadlock in Action

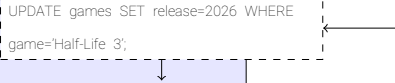
Transaction A (Serializable)

Transaction B (Serializable)

Deadlock in Action

Transaction A (Serializable)

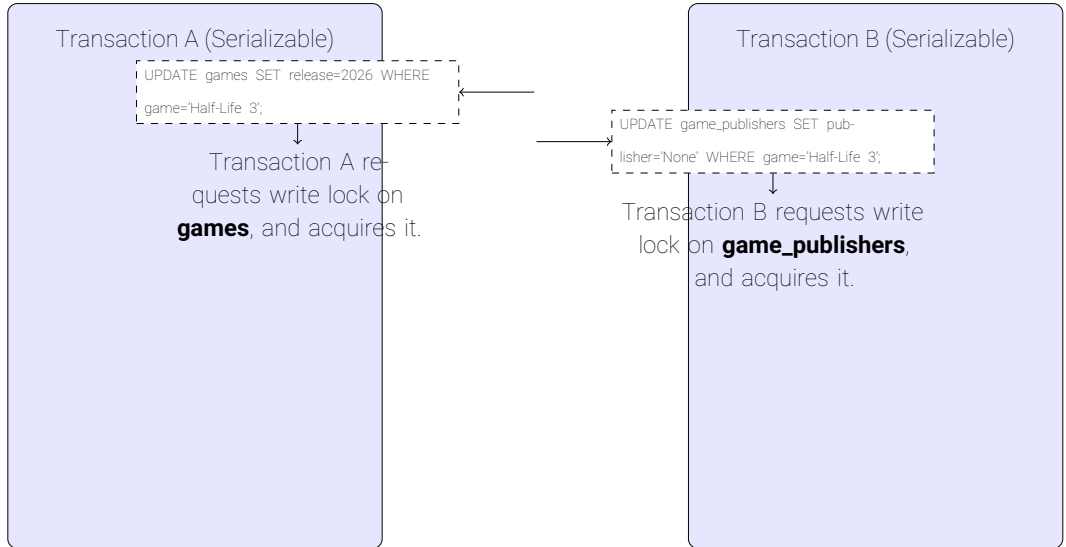
```
UPDATE games SET release=2026 WHERE  
game='Half-Life 3';
```



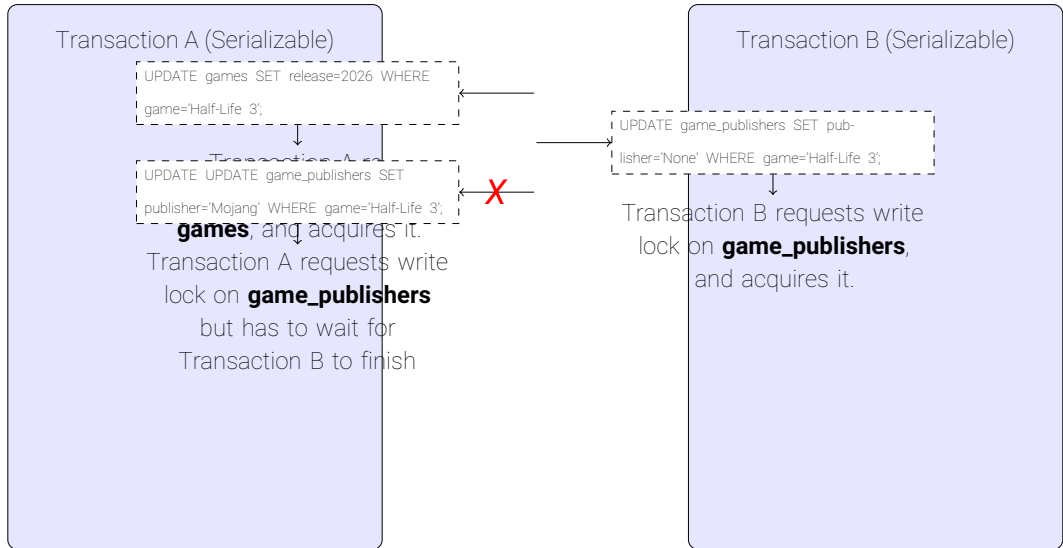
Transaction A re-
quests write lock on
games, and acquires it.

Transaction B (Serializable)

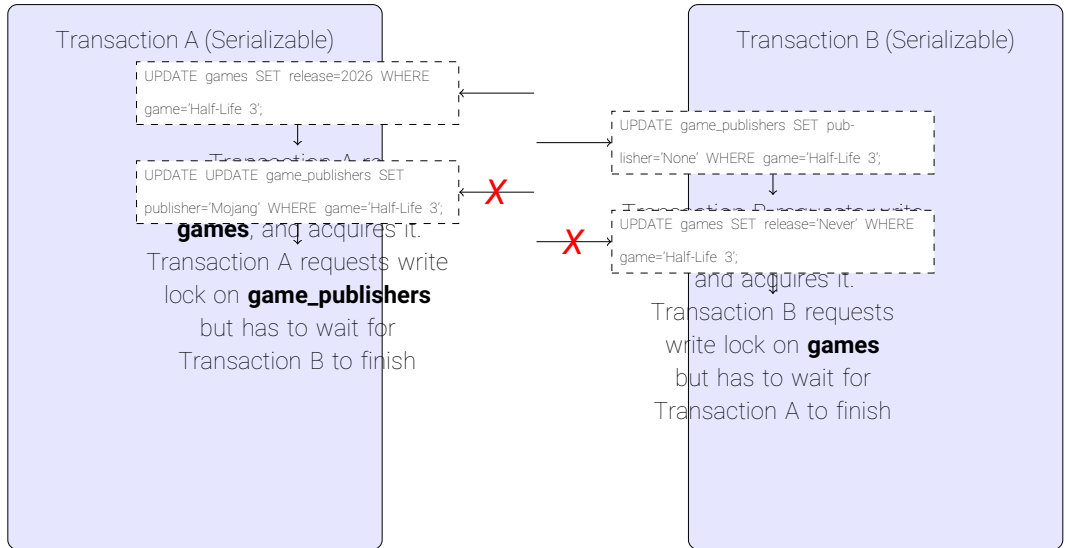
Deadlock in Action



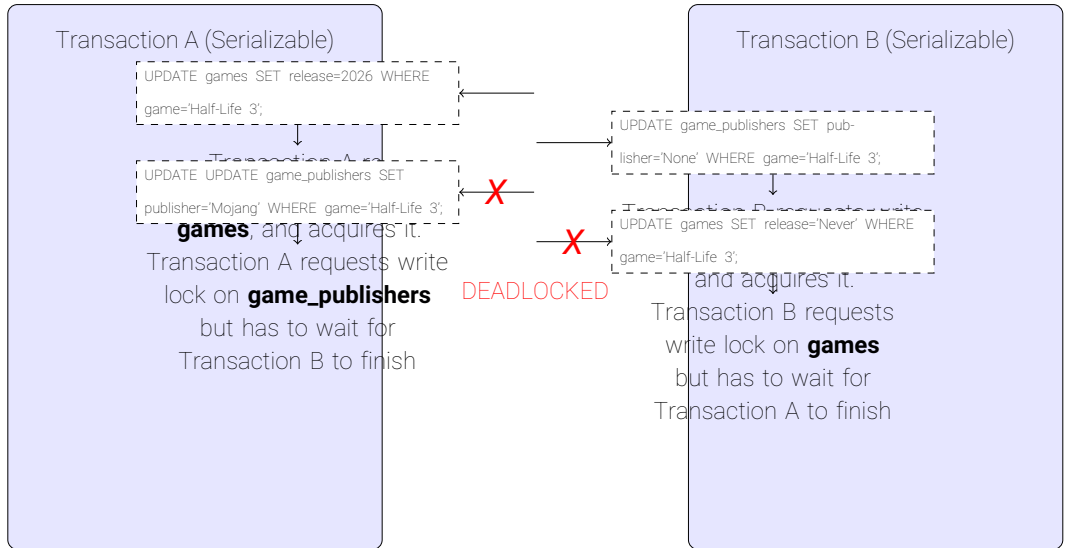
Deadlock in Action



Deadlock in Action



Deadlock in Action



Commands in Transactions

1	BEGIN TRANSACTION;	Starts a new transaction, some databases allow assigning names or defining read/write level with READ ONLY or READ WRITE
2		
3	sql statements	
4		Savepoints only exist for rolling back to this state in the transaction
5	SAVEPOINT name;	
6		resets to the state of a given savepoint name.
7	sql statements	ROLLBACK alone aborts the transaction completely.
8		
9	ROLLBACK TO name;	
10		Changes are made persistent only after
11	COMMIT;	COMMIT

Commands in Transactions - Example

Transaction that is successful:

```
1  BEGIN TRANSACTION;
2
3  INSERT INTO AchievementTypes
      VALUES (Collectibles, 22);
4  UPDATE AchievementTypes SET
      Points=40 WHERE
      AchievementType="Story";
5  SELECT * FROM AchievementTypes;
6
7  COMMIT;
```

Transaction that is aborted:

```
1  BEGIN TRANSACTION;
2
3  UPDATE games SET release=2026
      WHERE game='Half-Life 3';
4
5  ROLLBACK;
6
```

Advanced Transaction Handling

- **SET TRANSACTION** allows setting the Isolation Level, and READ/WRITE mode. See <https://dev.mysql.com/doc/refman/8.4/en/set-transaction.html>
 - This command works either on the entire database or a specific session.
- **LOCK TABLES** allows acquiring and releasing locks manually. See <https://dev.mysql.com/doc/refman/8.4/en/lock-tables.html>
 - Several databases allow locking data rows or entire tables but not all.
- Some commands (especially in the Data Definition Language) automatically commit and can NOT be rolled back. For example the **DROP TABLE** statement.
- Per default, commands that are not part of a transaction, are automatically handled as a transaction and committed by the database. This can be disabled. See <https://dev.mysql.com/doc/refman/8.4/en/commit.html>

Tips for Transaction Handling

- Keep Transactions short and fast. Longer transactions increase risk of deadlocks and slowing down the database.
- Use explicit TRANSACTIONS including CLOMMIT and ROLLBACKS.
- Use the right Isolation Level for your purpose. Sometimes Dirty Reads are not a problem, but sometimes you want Serializable.
- Use Savepoints so you don't have to rollback the entire transactions

Storage Engines

A storage engine is the component of a database management system (DBMS) that determine:

- data is stored and retrieved
- the data is managed on the disk or in memory
- supported features such as transaction handling
- trade offs in performance

Storage Engines in MariaDB

```
MariaDB [(none)]> SHOW ENGINES;
```

Engine	Support	Comment	Transactions	XA	Savepoints
CSV	YES	Stores tables as CSV files	NO	NO	NO
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
Aria	YES	Crash-safe tables with MyISAM heritage. Used for internal temporary tables and privilege tables	NO	NO	NO
MyISAM	YES	Non-transactional engine with good performance and small data footprint	NO	NO	NO
MRG_MyISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
InnoDB	DEFAULT	Supports transactions, row-level locking, foreign keys and encryption for tables	YES	YES	YES
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
SEQUENCE	YES	Generated tables filled with sequential values	YES	NO	YES

8 rows in set (0.000 sec)

Recommended Engines:

- InnoDB for transactions
- MyISAM for fast reads

InnoDB

- Default engine in MariaDB and MySQL
- Many features:
 - Transaction support including low level locking (rows in addition to tables)
 - Foreign keys
 - Full text search
 - Virtual Columns
 - Geospatial operations
- Performance and high complexity are a disadvantage

MyISAM

- Previous default in MySQL, replaced by InnoDB
- Fewer features than InnoDB
 - Not safe on crashes (breaking with ACID)
 - No transactions
- Very high performance, especially in applications with many reads and few writes
- Use strongly discouraged

Additional Engines

MariaDB supports many more engines for different purposes. An overview is given at

<https://mariadb.com/kb/en/choosing-the-right-storage-engine/>

- InnoDB is listed as general purpose
- Several distributed server applications exist, among them Spider, ColumnStore and TokuDB
- MyRocks works well for Archiving as it has good compression
- CONNECT is recommended to access files and remote data as if they were MariaDB Tables (including CSV)
- MEMORY is good for caching, but InnoDB already supports it out of the box. It can be used for Unit-Testing.

Indexing

- Indexes are used to improve read performance in databases
- An index is created on a table over one or more columns
- Indexes decrease write performance, and increase the memory footprint

Queries without indexing

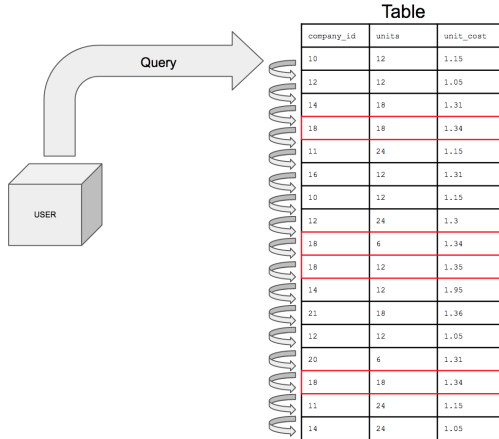


Figure: Without an index a sequential search needs to look through all rows sequentially.

Source: Atlassian

Queries with indexing

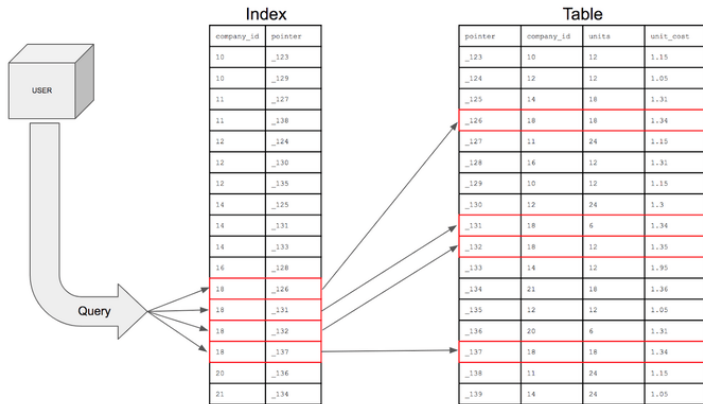


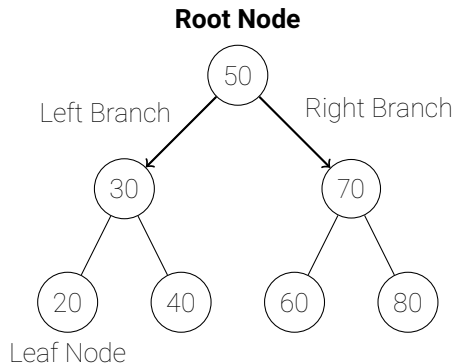
Figure: Via index, only relevant rows can be loaded. Source: Atlassian

<https://www.atlassian.com/data/databases/how-does-indexing-work>

How does the indexing work faster?

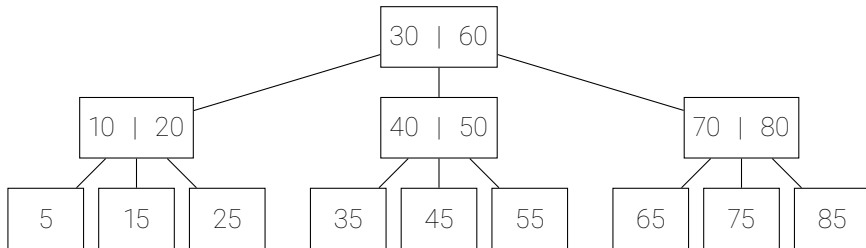
- An index is always sorted. (A-Z, numerical, ...)
- Indices are usually not lists, but data structures allowing fast modification and search
- Usually a B-Tree is used for indices

Binary Tree



- A B-Tree is always balanced. Each node has at most two children
- Each left child is $\leq$ parent, and each right child $\geq$ parent

B-Tree



- A B-Tree is an extension of a Binary Tree.
- Each node has at most m children, and at least $m/2$
- The tree is automatically balanced on each change

B+Tree

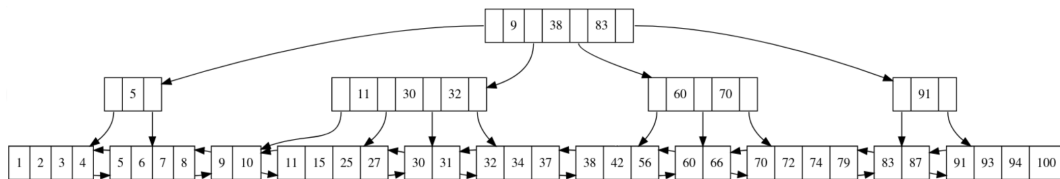


Figure: B+Tree. Generated via <https://roy2220.github.io/bptree/visualization/>

- A B+Tree is an extension of a B-Tree.
- All data is in the leaf nodes, which are also linked sequentially with each other.

Hash Index

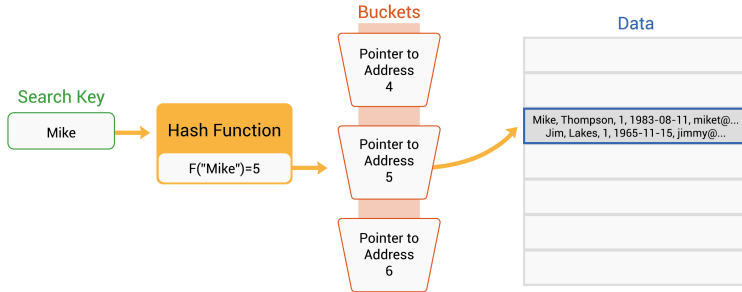
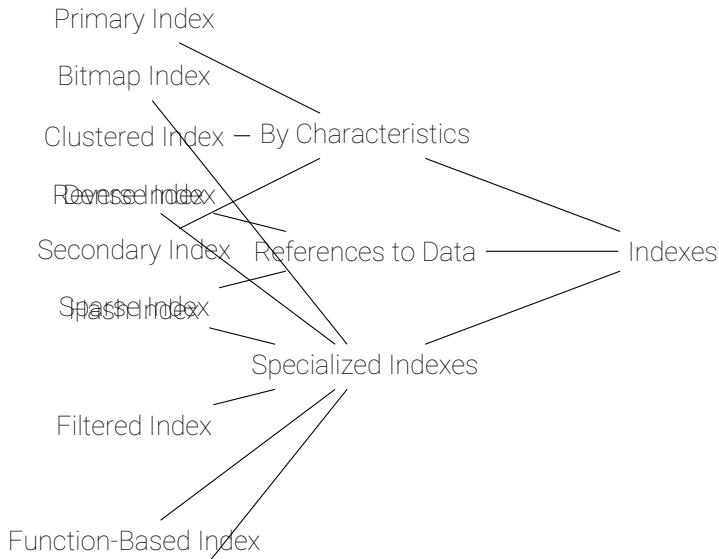


Figure: Hash Index. Source <https://vertabelo.com/blog/database-index-types/>

- A hash index is used to lookup unique values (exact search)
- Search keys are hashed to find the correct rows faster
- Used for primary keys and unique constraints

Other Index Types



Primary Key

- A primary key is also always an index
- It is always **unique**
- It can not be null
- A primary key value should not be changed anymore (but most DBs allow it)

Creating an Index

Will generate a simple index used to speedup search.

1 **CREATE INDEX name**

2 **ON table**

3 **(column1,**

4 **column2 DESC);**

Name of the Index. Some databases allow specializations (**CREATE BITMAP INDEX**, **CREATE CLUSTERED INDEX**, ...)

Table the index is created on

Column(s) that the index will be on.

Optionally ascending **ASC** or descending **DESC** can be defined on the column.

Create an Index Example

Index on title in the games table:

```
1  CREATE INDEX idx_games_title ON  
    games(title);
```

Speeds up this query:

```
1  SELECT * FROM games WHERE title =  
    'Half-Life 3';
```

Unique Index

- Will generate an index that must be unique over all columns.
- Allows enforcing UNIQUE constraint over multiple columns
- For a single column UNIQUE (see DDL) and UNIQUE INDEX are the same in (most) databases.
 - WARNING! Don't use both. Most databases will not identify the duplicate and create two separate indexes.

```
1  CREATE UNIQUE INDEX name
2  ON table
3  (column1, column2);
```

Ensures that the combination of columns in this table must be unique.

Example:

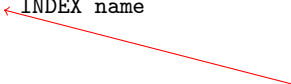
```
1  CREATE UNIQUE INDEX idx_games_title ON games(title);
```

Fulltext Search

- A **FULLTEXT** index speeds up <https://dev.mysql.com/doc/refman/8.4/en/fulltext-search.html>.
- WARNING! For large Datasets it is more efficient to first create and fill the table and then create the index afterwards.

```
1  CREATE FULLTEXT INDEX name
2  ON table
3  (column1);
```

Creates a fulltext index.



Example:

```
1  CREATE FULLTEXT INDEX idx_games_desc ON games(description);
```

Alternate Index Creation and Deletion

Index on table CREATE:

```
1  CREATE TABLE games (  
2      id INT PRIMARY KEY  
      AUTO_INCREMENT,  
3      title VARCHAR(255) NOT NULL,  
4      description VARCHAR(5000) NOT  
      NULL,  
5      INDEX idx_games_description (  
      title)  
6  );
```

Index on table ALTER:

```
1  ALTER TABLE games ADD INDEX  
      idx_games_title_date (title,  
      release_date);
```

Drop Index:

```
1  ALTER TABLE games DROP INDEX  
      idx_games_title_date;
```


When to use an index

Advantages of an index:

- Improved data access. E.g. on read heavy data
- Faster query performance if the column(s) are often used in queries.
- Optimized sorting if data is queried ASC or DESC

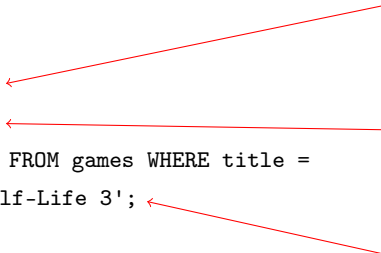
Avoid indexes:

- When data is manipulated often, as these operations become more expensive due to index maintenance
- Columns are rarely queried should not have indexes
- Small tables usually don't need indexes

Explaining SQL Statements

Will **EXPLAIN** a table structure or query.

```
1  EXPLAIN  
2  ANALYZE  
3  SELECT * FROM games WHERE title =  
    'Half-Life 3';
```



Explains the following statement or table. In many databases synonymous with **DESCRIBE**. Only provides estimates on performance.

Optional, executes queries and gives performance statistics on how many rows are loaded and how long it took.

Statement to be explained and/or analyzed

The power of indices

```
EXPLAIN SELECT * FROM games WHERE metacritic > 50
```

Without index:

```
1 { "query_block": {  
2   "select_id": 1,  
3   "cost": 12.0306708,  
4   "nested_loop": [{  
5     "table": {  
6       "table_name": "games",  
7       "access_type": "ALL",  
8       "loops": 1,  
9       "rows": 71224,  
10      "cost": 12.0306708,  
11      "filtered": 100,  
12      "attached_condition": "games.metacritic > 50"  
13    }  
14  ]  
15 }  
16 }  
17 }
```

```
CREATE INDEX metacritic_index ON games(  
metacritic);
```

```
1 { "query_block": {  
2   "select_id": 1,  
3   "cost": 2.81237968,  
4   "nested_loop": [{  
5     "table": {  
6       "table_name": "games",  
7       "access_type": "range",  
8       "possible_keys": ["metacritic_index"],  
9       "key": "metacritic_index",  
10      "key_length": "5",  
11      "used_key_parts": ["metacritic"],  
12      "loops": 1,  
13      "rows": 2431,  
14      "cost": 2.81237968,  
15      "filtered": 100,  
16      "index_condition": "games.metacritic > 50"  
17    }  
18  ]  
19 }  
20 }
```

The power of indices

```
ANALYZE SELECT * FROM games WHERE metacritic > 50
```

Without index:

```
1 { "query_optimization": {
2   "r_total_time_ms": 0.051408534},
3   "query_block": {
4     "select_id": 1,
5     "cost": 12.0306708,
6     "r_loops": 1,
7     "r_total_time_ms": 14.23644847,
8     "nested_loop": [{
9       "table": {
10        "table_name": "games",
11        "access_type": "ALL",
12        "loops": 1,
13        "r_loops": 1,
14        "rows": 71224,
15        "r_rows": 71429,
16        "cost": 12.0306708,
17        "r_table_time_ms": 12.94784096,
18        "r_other_time_ms": 1.283230339,
19        "r_engine_stats": { "pages_accessed": 465 },
20        "filtered": 100,
21        "r_total_filtered": 3.40337958,
22        "attached_condition": "games.metacritic > 50",
23        "r_filtered": 3.40337958
24      }
25    }
26  }
27 }
```

```
CREATE INDEX metacritic_index ON games(
metacritic);
```

```
1 { "query_optimization": {
2   "r_total_time_ms": 0.076974643},
3   "query_block": {
4     "select_id": 1,
5     "cost": 2.81237968,
6     "r_loops": 1,
7     "r_total_time_ms": 2.310717105,
8     "nested_loop": [{
9       "table": {
10        "table_name": "games",
11        "access_type": "range",
12        "possible_keys": ["metacritic_index"],
13        "key": "metacritic_index",
14        "key_length": "5",
15        "used_key_parts": ["metacritic"],
16        "loops": 1,
17        "r_loops": 1,
18        "rows": 2431,
19        "r_index_rows": 2431,
20        "r_rows": 2431,
21        "cost": 2.81237968,
22        "r_table_time_ms": 2.140263777,
23        "r_other_time_ms": 0.164311139,
24        "r_engine_stats": { "pages_accessed": 4866 },
25        "filtered": 100,
26        "r_total_filtered": 100,
27        "index_condition": "games.metacritic > 50",
28        "r_icp_filtered": 100,
29        "r_filtered": 100
30      }
31    }
32  }
33 }
```